Liblog User Guide

Gustavo Bacagine
2023

Liblog User Guide

Gustavo Bacagine
2023

Table of Contents

1 INTRODUCTION....4
1.1 UNDERSTANTING THE PROGRAMMING METHOD USED IN THIS FILE....4
1.1.1 Hungarian Notation Usage Example....4
2 DEFINES AND MACROS....6
2.1 DEBUG LEVELS....6
2.2 COLOR DEFINES....6
2.3 FUNCTIONS MACROS....6
3 TYPEDEFS....8
4 VARIABLES....9
5 FUNCTIONS....10
5.1 SETTERS....10
5.2 GETTERS....10
5.3 PRINT TRACE MESSAGE....10
APENDIX A – SOURCE CODE....11
log.h....11
loc.c....14
APENDIX B – USAGE EXAMPLES....23
Example 1 - colored.c....23
Example 2 debug.c....24
Example 3 - hello_log.c....27
Example 4 - trace.c....27
Example 5 - trace_level.c....29
Example 6 - Modular programming....30
module1.c:....30
module2.c:....31
APENDIX C – LICENSE....34
GNU GENERAL PUBLIC LICENSE Version 3, 29 June 2007....34
PREAMBLE....34
TERMS AND CONDITIONS....35
0. Definitions....35
1. Source Code....36
2. Basic Permissions....37
3. Protecting Users' Legal Rights From Anti-Circumvention Law....37
4. Conveying Verbatim Copies....37
5. Conveying Modified Source Versions....37
6. Conveying Non-Source Forms....38
7. Additional Terms....40
8. Termination....41
9. Acceptance Not Required for Having Copies....41
10. Automatic Licensing of Downstream Recipients....41
11. Patents....42
12. No Surrender of Others' Freedom....43
13. Use with the GNU Affero General Public License....43
14. Revised Versions of this License....43
15. Disclaimer of Warranty....44
16. Limitation of Liability....44
17. Interpretation of Sections 15 and 16....44
END OF TERMS AND CONDITIONS....44
How to Apply These Terms to Your New Programs....44

1 INTRODUCTION

The liblog is a C trace library developed by Gustavo Bacagine <gustavo.bacagine@protonmail.com>.

This library is a free software (see apendix C to read the license)

1.1 UNDERSTANTING THE PROGRAMMING METHOD USED IN THIS FILE

In my programs I use a customized version of Hungarian Notation.

Hungarian notationaims to facilitate recognition of the type of variable in a program, just by looking at the name of the variable. See below a table with the type of indicators for each type of data.

Indicator	Type
g	Global
k	Constant
a	Array
p	Pointer
v	Void
b	Boolean
ch	Char
i	Integer
f	Fload
d	Double
sz	String
st	Struct
u	Union
e	Enum
fp	File Pointer

1.1.1 Hungarian Notation Usage Example

```
typedef struct STRUCT_PERSON
{
  int iID;
  char szName[64];
  char cSex;
  float fHeight;
  double dSalary;
```

```
  bool bLive;
} STRUCT_PERSON, *PSTRUCT_PERSON

const double gkdPI = 3.14;

...

int main(int argc, char **argv)
{
  PSTRUCT_PERSON pstPerson = (PSTRUCT_PERSON) malloc(sizeof(STRUCT_PERSON);

  ...

  free(pstPerson);
  pstPerson = NULL;

  return 0;
}
```

2 DEFINES AND MACROS

In this chapter I explain the defines and macros defined in log.h

2.1 DEBUG LEVELS

```
#define INFO_LEVEL    0
#define WARNING_LEVEL 1
#define ERROR_LEVEL   2
#define FATAL_LEVEL   3
#define DEBUG_LEVEL   4
#define TRACE_LEVEL   5
```

INFO_LEVEL (0): Normal messages

WARNING_LEVEL (1): Alert messages

ERROR_LEVEL (2): Errors that not cause the interruption of program

FATAL_LEVEL (3): Erros that cause the interruption of program

DEBUG_LEVEL (4): Detailed messages intended for the programmer

TRACE_LEVEL (5): Messages more detailed than DEBUG_LEVEL level. Show all log level messages

2.2 COLOR DEFINES

```
/* White color */
#define INFO_INIT_COLOR "\033[1;37m"
#define INFO_END_COLOR  "\033[0m"

/* Cyan color */
#define WARNING_INIT_COLOR "\033[1;35m"
#define WARNING_END_COLOR  "\033[0m"

/* Red color */
#define ERROR_INIT_COLOR "\033[1;31m"
#define ERROR_END_COLOR  "\033[0m"

#define FATAL_INIT_COLOR "\033[1;31m"
#define FATAL_END_COLOR  "\033[0m"

/* Blue color */
#define DEBUG_INIT_COLOR "\033[1;34m"
#define DEBUG_END_COLOR  "\033[0m"

/* Yellow color */
#define TRACE_INIT_COLOR "\033[1;33m"
#define TRACE_END_COLOR  "\033[0m"
```

This macors is used in function _vTrace.

2.3 FUNCTIONS MACROS

```
#define vTraceInfo(FORMAT, ...)    _vTrace(INFO_LEVEL,    __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
#define vTraceWarning(FORMAT, ...) _vTrace(WARNING_LEVEL, __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
```

```
#define vTraceError(FORMAT, ...)   _vTrace(ERROR_LEVEL,   __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
#define vTraceFatal(FORMAT, ...)   _vTrace(FATAL_LEVEL,   __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
#define vTraceDebug(FORMAT, ...)   _vTrace(DEBUG_LEVEL,   __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
#define vTraceAll(FORMAT, ...)     _vTrace(TRACE_LEVEL,   __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)

#define vTrace(LOG_LEVEL, FORMAT, ...) if(giDebugLevel >= LOG_LEVEL)
_vTrace(LOG_LEVEL, __FILE__, __LINE__, FORMAT, ##__VA_ARGS__)

#define vTraceBegin() if(INFO_LOG_DETAILS) _vTrace(INFO_LEVEL, __FILE__,
__LINE__, "%s - begin", __func__)
#define vTraceEnd()   if(INFO_LOG_DETAILS) _vTrace(INFO_LEVEL, __FILE__,
__LINE__, "%s - end", __func__)
```

The first six defines is used to print a message with a specific debug level.

The macro vTrace is used to print a message with a debug level passed for the developer.

The vtraceBegin is used in the beginning of a function, and vTraceEnd is used in the end of a funciton.

Example:

```
void vFunc(void)
{
  vTraceBegin();

  if(INFO_LOG_DETAILS) vTrace("Hello World!!!");

  vTraceEnd();
}
```

3 TYPEDEFS

```
typedef int DebugLevel;
```

Is the type of debug level variable

4 VARIABLES

```
static const char *kszLogLevel[] = {
  "INFO", "WARNING", "ERROR", "FATAL", "DEBUG", "TRACE"
};
```

Used to show debug level in .log file

```
static const char *kszLogLevelColorInit[] = {
  INFO_INIT_COLOR, WARNING_INIT_COLOR,
  ERROR_INIT_COLOR, FATAL_INIT_COLOR,
  DEBUG_INIT_COLOR, TRACE_INIT_COLOR
};

static const char *kszLogLevelColorEnd[] = {
  INFO_END_COLOR, WARNING_END_COLOR,
  ERROR_END_COLOR, FATAL_END_COLOR,
  DEBUG_END_COLOR, TRACE_END_COLOR
};
```

The arrays kszLogLevelColorInit and kszLogLevelColorEnd are used to print a colored log level message in .log file

```
extern char gszConfFileName[LOG_FILE_NAME_LENGTH];
```

Used to save the name of .conf file

```
extern char gszLogFileName[LOG_FILE_NAME_LENGTH];
```

Used to save the name of .log file

```
extern DebugLevel giDebugLevel;
```

Used to save the debug level

5 FUNCTIONS

5.1 SETTERS

```
void vSetConfFileName(const char *kszConfFile);
```

Set the name of configuration file, see bellow a example of .conf file:

```
#
# Test for my logger software in Written in C
#

# 0 to 5
LOG_LEVEL = 5

# true or false
COLORED_LOG_LEVEL = true
```

```
void vSetLogFileName(const char *kszLogFileName);
```

Set the name of .log file

```
void vSetColoredLogLevel(bool bColored);
```

Set the value of `gbColoredLogLevel` variable

```
void vSetLogLevel(DebugLevel usiDebugLevel);
```

Set the value of `giDebugLevel` variable

5.2 GETTERS

```
int iGetLogLevel(void);
```

Get the value of `giDebugLevel` variable

```
int iGetColoredLogLevel(void);
```

Get the value of `gbColoredLogLevel` variable

5.3 PRINT TRACE MESSAGE

```
void _vTrace(const DebugLevel usiDebugLevel,
             const char *kpszModuleName,
             const int kiLine,
             const char *kpszFmt, ...);
```

Print a message in the .log file

APENDIX A – SOURCE CODE

log.h

```
/* log.h: This file contains constants and functions to create
 * .log files
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmai.com>
 *
 * Date: 2023-06-16
 */

#ifndef _LOG_H_
#define _LOG_H_

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <stdarg.h>
#include <time.h>
#include <errno.h>

#define CONF_FILE_NAME_LENGTH 256
#define LOG_FILE_NAME_LENGTH  256

#define INFO_LEVEL    0 /* Normal messages
*/
#define WARNING_LEVEL 1 /* Warning alert messages
*/
#define ERROR_LEVEL   2 /* Error messages
*/
#define FATAL_LEVEL   3 /* Fatal error messages
*/
#define DEBUG_LEVEL   4 /* Messages with more information about the source code
*/
#define TRACE_LEVEL   5 /* Show all messages in log
*/

/**
 * Example of use:
 *
 * if(INFO_LOG_DETAILS)
 * {
 *   vTraceInfo("INFO Message");
 * }
 *
 * if(DEBUG_LOG_DETAILS)
 * {
 *   vTraceDebug("DEBUG Message");
 * }
 */
#define INFO_LOG_DETAILS    giDebugLevel >= INFO_LEVEL
#define WARNING_LOG_DETAILS giDebugLevel >= WARNING_LEVEL
#define ERROR_LOG_DETAILS   giDebugLevel >= ERROR_LEVEL
#define FATAL_LOG_DETAILS   giDebugLevel >= FATAL_LEVEL
#define DEBUG_LOG_DETAILS   giDebugLevel >= DEBUG_LEVEL
#define TRACE_LOG_DETAILS   giDebugLevel >= TRACE_LEVEL

/* White color */
#define INFO_INIT_COLOR "\033[1;37m"
```

```
#define INFO_END_COLOR  "\033[0m"

/* Cyan color */
#define WARNING_INIT_COLOR "\033[1;35m"
#define WARNING_END_COLOR  "\033[0m"

/* Red color */
#define ERROR_INIT_COLOR "\033[1;31m"
#define ERROR_END_COLOR  "\033[0m"

#define FATAL_INIT_COLOR "\033[1;31m"
#define FATAL_END_COLOR  "\033[0m"

/* Blue color */
#define DEBUG_INIT_COLOR "\033[1;34m"
#define DEBUG_END_COLOR  "\033[0m"

/* Yellow color */
#define TRACE_INIT_COLOR "\033[1;33m"
#define TRACE_END_COLOR  "\033[0m"

#define vTraceInfo(FORMAT, ...)    _vTrace(INFO_LEVEL,    __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
#define vTraceWarning(FORMAT, ...) _vTrace(WARNING_LEVEL, __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
#define vTraceError(FORMAT, ...)   _vTrace(ERROR_LEVEL,   __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
#define vTraceFatal(FORMAT, ...)   _vTrace(FATAL_LEVEL,   __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
#define vTraceDebug(FORMAT, ...)   _vTrace(DEBUG_LEVEL,   __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)
#define vTraceAll(FORMAT, ...)     _vTrace(TRACE_LEVEL,   __FILE__, __LINE__,
FORMAT, ##__VA_ARGS__)

#define vTrace(LOG_LEVEL, FORMAT, ...) if(giDebugLevel >= LOG_LEVEL)
_vTrace(LOG_LEVEL, __FILE__, __LINE__, FORMAT, ##__VA_ARGS__)

#define vTraceBegin() if(INFO_LOG_DETAILS) _vTrace(INFO_LEVEL, __FILE__,
__LINE__, "%s - begin", __func__)
#define vTraceEnd()   if(INFO_LOG_DETAILS) _vTrace(INFO_LEVEL, __FILE__,
__LINE__, "%s - end", __func__)

typedef int DebugLevel;

static const char *kszLogLevel[] = {
  "INFO", "WARNING", "ERROR", "FATAL", "DEBUG", "TRACE"
};

static const char *kszLogLevelColorInit[] = {
  INFO_INIT_COLOR, WARNING_INIT_COLOR,
  ERROR_INIT_COLOR, FATAL_INIT_COLOR,
  DEBUG_INIT_COLOR, TRACE_INIT_COLOR
};

static const char *kszLogLevelColorEnd[] = {
  INFO_END_COLOR, WARNING_END_COLOR,
  ERROR_END_COLOR, FATAL_END_COLOR,
  DEBUG_END_COLOR, TRACE_END_COLOR
};

/**
 * Receive the name of .conf file
 */
extern char gszConfFileName[LOG_FILE_NAME_LENGTH];
```

```
/**
 * Receive the name of .log file
 */
extern char gszLogFileName[LOG_FILE_NAME_LENGTH];

/**
 * Receive the level of debug from .conf
 * file, the default is 0 (INFO_LEVEL)
 */
extern DebugLevel giDebugLevel;

/**
 * Receive if the log level tag is colored
 * or not, the default is without color
 */
extern bool gbColoredLogLevel;

/**
 * Set the name of configure file of
 * the software
 */
void vSetConfFileName(const char *kszConfFile);

/**
 * Set the name of log file
 */
void vSetLogFileName(const char *kszLogFileName);

/**
 * Enable or disable log level with
 * colored and bold
 */
void vSetColoredLogLevel(bool bColored);

/**
 * Set the value of giDebugLevel
 */
void vSetLogLevel(DebugLevel usiDebugLevel);

/**
 * Get the log level from file.
 *
 * return values:
 * 0 - 6 = Log Level
 * -1 if can't open the file.
 * -2 if LOG_LEVEL is empty in file
 * -3 if don't found LOG_LEVEL variable in file
 * -4 if value of LOG_LEVEL is invalid
 */
int iGetLogLevel(void);

/**
 * Get the colored log level from file.
 *
 * return values:
 *  0 is fals, no color in log level
 *  1 is true, colored in log level
 * -1 if can't open the file.
 * -2 if COLORED_LOG_LEVEL is empty in file
 * -3 if don't found COLORED_LOG_LEVEL variable in file
 * -4 if value of COLORED_LOG_LEVEL is invalid
 */
int iGetColoredLogLevel(void);
```

```
/**
 * Print a log message with the log level between [].
 */
void _vTrace(const DebugLevel usiDebugLevel,
             const char *kpszModuleName,
             const int kiLine,
             const char *kpszFmt, ...);

#endif /* _LOG_H_ */
```

loc.c

```
/* log.c: This file contains the implementation of logger functions
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-06-16
 */

#include "log.h"

char gszConfFileName[LOG_FILE_NAME_LENGTH];
char gszLogFileName[LOG_FILE_NAME_LENGTH];
DebugLevel giDebugLevel = 0;
bool gbColoredLogLevel = false;

void vSetConfFileName(const char *kszConfFileName)
{
  if(sizeof(kszConfFileName) > sizeof(gszConfFileName))
  {
    fprintf(stderr, "E: The name of configuraation file is bigger than %d",
CONF_FILE_NAME_LENGTH);
    exit(EXIT_FAILURE);
  }

  snprintf(gszConfFileName, sizeof(gszConfFileName), "%s", kszConfFileName);
}

void vSetLogFileName(const char *kszLogFileName)
{
  if(sizeof(kszLogFileName) > sizeof(gszLogFileName))
  {
    fprintf(stderr, "E: The name of log file is bigger than %d",
LOG_FILE_NAME_LENGTH);
    exit(EXIT_FAILURE);
  }

  snprintf(gszLogFileName, sizeof(gszLogFileName), "%s", kszLogFileName);
}

void vSetColoredLogLevel(bool bColored)
{
  gbColoredLogLevel = bColored;
}

void vSetLogLevel(DebugLevel usiDebugLevel)
{
  giDebugLevel = usiDebugLevel;
}

void _vTrace(DebugLevel usiDebugLevel,
```

```
            const char *kpszModuleName,
            const int kiLine,
            const char *kpszFmt, ...)
{
  va_list args;
  FILE *fpLogFile = NULL;
  time_t tTime = time(NULL);
  struct tm *stTime = localtime(&tTime);
  char szDbg[2048];

  memset(szDbg , 0, sizeof(szDbg));

  if((fpLogFile = fopen(gszLogFileName, "a")) == NULL)
  {
    fprintf(stderr, "E: Impossible create or open file %s!\n"
                    "%s\n", gszLogFileName, strerror(errno));

    exit(EXIT_FAILURE);
  }

  va_start(args, kpszFmt);

  snprintf(szDbg, sizeof(szDbg), "%04d/%02d/%02d %02d:%02d:%02d %s:%d", stTime-
>tm_year + 1900,
                                                                        stTime-
>tm_mon + 1,
                                                                        stTime-
>tm_mday,
                                                                        stTime-
>tm_hour,
                                                                        stTime-
>tm_min,
                                                                        stTime-
>tm_sec,

kpszModuleName,
                                                                        kiLine
  );

  switch(usiDebugLevel)
  {
    case INFO_LEVEL:
      if(gbColoredLogLevel)
      {
        char szLogLevel[22];

        memset(szLogLevel, 0, sizeof(szLogLevel));

        strcat(szLogLevel, kszLogLevelColorInit[INFO_LEVEL]);
        strcat(szLogLevel, kszLogLevel[INFO_LEVEL]);
        strcat(szLogLevel, kszLogLevelColorEnd[INFO_LEVEL]);

        strcat(szDbg, " [");
        strcat(szDbg, szLogLevel);
        strcat(szDbg, "] ");

        strcat(szDbg, kpszFmt);
        strcat(szDbg, "\n");

        vfprintf(fpLogFile, szDbg, args);
        break;
      }

      strcat(szDbg, " [");
```

```
    strcat(szDbg, kszLogLevel[INFO_LEVEL]);
    strcat(szDbg, "] ");

    strcat(szDbg, kpszFmt);
    strcat(szDbg, "\n");

    vfprintf(fpLogFile, szDbg, args);
    break;
  case WARNING_LEVEL:
    if(gbColoredLogLevel)
    {
      char szLogLevel[25];

      memset(szLogLevel, 0, sizeof(szLogLevel));

      strcat(szLogLevel, kszLogLevelColorInit[WARNING_LEVEL]);
      strcat(szLogLevel, kszLogLevel[WARNING_LEVEL]);
      strcat(szLogLevel, kszLogLevelColorEnd[WARNING_LEVEL]);

      strcat(szDbg, " [");
      strcat(szDbg, szLogLevel);
      strcat(szDbg, "] ");

      strcat(szDbg, kpszFmt);
      strcat(szDbg, "\n");

      vfprintf(fpLogFile, szDbg, args);
      break;
    }

    strcat(szDbg, " [");
    strcat(szDbg, kszLogLevel[WARNING_LEVEL]);
    strcat(szDbg, "] ");

    strcat(szDbg, kpszFmt);
    strcat(szDbg, "\n");

    vfprintf(fpLogFile, szDbg, args);
    break;
  case ERROR_LEVEL:
    if(gbColoredLogLevel)
    {
      char szLogLevel[23];

      memset(szLogLevel, 0, sizeof(szLogLevel));

      strcat(szLogLevel, kszLogLevelColorInit[ERROR_LEVEL]);
      strcat(szLogLevel, kszLogLevel[ERROR_LEVEL]);
      strcat(szLogLevel, kszLogLevelColorEnd[ERROR_LEVEL]);

      strcat(szDbg, " [");
      strcat(szDbg, szLogLevel);
      strcat(szDbg, "] ");

      strcat(szDbg, kpszFmt);
      strcat(szDbg, "\n");

      vfprintf(fpLogFile, szDbg, args);
      break;
    }

    strcat(szDbg, " [");
    strcat(szDbg, kszLogLevel[ERROR_LEVEL]);
    strcat(szDbg, "] ");
```

```
    strcat(szDbg, kpszFmt);
    strcat(szDbg, "\n");

    vfprintf(fpLogFile, szDbg, args);
    break;
  case FATAL_LEVEL:
    if(gbColoredLogLevel)
    {
      char szLogLevel[23];

      memset(szLogLevel, 0, sizeof(szLogLevel));

      strcat(szLogLevel, kszLogLevelColorInit[FATAL_LEVEL]);
      strcat(szLogLevel, kszLogLevel[FATAL_LEVEL]);
      strcat(szLogLevel, kszLogLevelColorEnd[FATAL_LEVEL]);

      strcat(szDbg, " [");
      strcat(szDbg, szLogLevel);
      strcat(szDbg, "] ");

      strcat(szDbg, kpszFmt);
      strcat(szDbg, "\n");

      vfprintf(fpLogFile, szDbg, args);
      break;
    }

    strcat(szDbg, " [");
    strcat(szDbg, kszLogLevel[FATAL_LEVEL]);
    strcat(szDbg, "] ");

    strcat(szDbg, kpszFmt);
    strcat(szDbg, "\n");

    vfprintf(fpLogFile, szDbg, args);
    break;
  case DEBUG_LEVEL:
    if(gbColoredLogLevel)
    {
      char szLogLevel[23];

      memset(szLogLevel, 0, sizeof(szLogLevel));

      strcat(szLogLevel, kszLogLevelColorInit[DEBUG_LEVEL]);
      strcat(szLogLevel, kszLogLevel[DEBUG_LEVEL]);
      strcat(szLogLevel, kszLogLevelColorEnd[DEBUG_LEVEL]);

      strcat(szDbg, " [");
      strcat(szDbg, szLogLevel);
      strcat(szDbg, "] ");

      strcat(szDbg, kpszFmt);
      strcat(szDbg, "\n");

      vfprintf(fpLogFile, szDbg, args);
      break;
    }

    strcat(szDbg, " [");
    strcat(szDbg, kszLogLevel[DEBUG_LEVEL]);
    strcat(szDbg, "] ");

    strcat(szDbg, kpszFmt);
```

```
      strcat(szDbg, "\n");

      vfprintf(fpLogFile, szDbg, args);
      break;
    case TRACE_LEVEL:
      if(gbColoredLogLevel)
      {
        char szLogLevel[23];

        memset(szLogLevel, 0, sizeof(szLogLevel));

        strcat(szLogLevel, kszLogLevelColorInit[TRACE_LEVEL]);
        strcat(szLogLevel, kszLogLevel[TRACE_LEVEL]);
        strcat(szLogLevel, kszLogLevelColorEnd[TRACE_LEVEL]);

        strcat(szDbg, " [");
        strcat(szDbg, szLogLevel);
        strcat(szDbg, "] ");

        strcat(szDbg, kpszFmt);
        strcat(szDbg, "\n");

        vfprintf(fpLogFile, szDbg, args);
        break;
      }

      strcat(szDbg, " [");
      strcat(szDbg, kszLogLevel[TRACE_LEVEL]);
      strcat(szDbg, "] ");

      strcat(szDbg, kpszFmt);
      strcat(szDbg, "\n");

      vfprintf(fpLogFile, szDbg, args);
      break;
    default:
      break;
  }

  va_end(args);

  fclose(fpLogFile);
  fpLogFile = NULL;
}

int iGetLogLevel(void)
{
  FILE *fpLogConfFile = NULL;
  int iDebugLevel = -1;
  char szLine[256];
  int iLineLen = sizeof(szLine);

  /* Used to verify if the LOG_LEVEL
   * variable is found in .conf file */
  bool bFoundLogLevel = false;

  char *pToken = NULL;
  int ii = 0;
  char szLevel[2];

  memset(szLine, 0, iLineLen);
  memset(szLevel, 0, sizeof(szLevel));

  /* Try open .conf file */
```

```
  if((fpLogConfFile = fopen(gszConfFileName, "r")) == NULL)
  {
    fprintf(stderr, "E: %s %s\n", gszConfFileName, strerror(errno));

    return -1;
  }

  while(fgets(szLine, iLineLen, fpLogConfFile) != NULL)
  {
    /* Ignore commented szLines */
    if(strstr(szLine, "#")) continue;
    pToken = strtok(szLine, " = ");

    if(strstr(szLine, "LOG_LEVEL"))
    {
      bFoundLogLevel = true;

      while(pToken != NULL)
      {
        /* Getting the value after '='
         * symbol and converting form
         * string to char
         */
        if(ii == 1)
        {
          /* Verify if have a value after '=' symbol */
          if(!strcmp(pToken, "") || !strcmp(pToken, " ") || !strcmp(pToken, "\
n"))
          {
            fprintf(stderr, "E: LOG_LEVEL is empty in file %s!\n",
gszConfFileName);

            fclose(fpLogConfFile);
            fpLogConfFile = NULL;

            return -2;
          }

          snprintf(szLevel, sizeof(szLevel), "%s", pToken);
          iDebugLevel = atoi(szLevel);

          break;
        }
        pToken = strtok(NULL, " = ");

        ii++;
      }

      break;
    }
    else
    {
      continue;
    }
  }

  /* If don't found the LOG_LEVEL
   * variable in file print a error
   * message to user */
  if(bFoundLogLevel == false)
  {
    fprintf(stderr, "E: Not found variable LOG_LEVEL in file %s!\n",
gszConfFileName);
```

```
    fclose(fpLogConfFile);
    fpLogConfFile = NULL;

    return -3;
  }

  /* If LOG_LEVEL in file is incorret */
  if(iDebugLevel < 0 || iDebugLevel > 5)
  {
    fprintf(stderr, "E: Invalid value of log level in file %s!\n",
gszConfFileName);

    fclose(fpLogConfFile);
    fpLogConfFile = NULL;

    return -4;
  }

  fclose(fpLogConfFile);
  fpLogConfFile = NULL;

  return iDebugLevel;
}

int iGetColoredLogLevel(void)
{
  FILE *fpLogConfFile = NULL;
  char szLine[256];
  int iLineLen = sizeof(szLine);

  /* Used to verify if the COLORED_LOG_LEVEL
   * variable is found in .conf file */
  bool bFoundColoredLogLevel = false;

  char *pToken = NULL;
  int ii = 0;
  char szBool[6]; /* Receive true or false */
  int iReturnValue = false;

  memset(szLine, 0, iLineLen);
  memset(szBool, 0, sizeof(szBool));

  /* Try open .conf file */
  if((fpLogConfFile = fopen(gszConfFileName, "r")) == NULL)
  {
    fprintf(stderr, "E: %s %s\n", gszConfFileName, strerror(errno));

    return -1;
  }

  while(fgets(szLine, iLineLen, fpLogConfFile) != NULL)
  {
    /* Ignore commented szLines */
    if(strstr(szLine, "#")) continue;
    pToken = strtok(szLine, " = ");

    if(strstr(szLine, "COLORED_LOG_LEVEL"))
    {
      bFoundColoredLogLevel = true;

      while(pToken != NULL)
      {
        /* Getting the value after '='
         * symbol and converting form
```

```
         * string to char
         */
        if(ii == 1)
        {
          /* Verify if have a value after '=' symbol */
          if(!strcmp(pToken, "") || !strcmp(pToken, " ") || !strcmp(pToken, "\
n"))
          {
            fprintf(stderr, "E: COLORED_LOG_LEVEL is empty in file %s!\n",
gszConfFileName);

            fclose(fpLogConfFile);
            fpLogConfFile = NULL;

            return -2;
          }

          snprintf(szBool, sizeof(szBool), "%s", pToken);
          szBool[strcspn(szBool, "\n")] = 0;

          break;
        }
        pToken = strtok(NULL, " = ");

        ii++;
      }

      break;
    }
    else
    {
      continue;
    }
  }

  /* If don't found the LOG_LEVEL
   * variable in file print a error
   * message to user */
  if(bFoundColoredLogLevel == false)
  {
    fprintf(stderr, "E: Not found variable COLORED_LOG_LEVEL in file %s!\n",
gszConfFileName);

    fclose(fpLogConfFile);
    fpLogConfFile = NULL;

    return -3;
  }

  /* If COLORED_LOG_LEVEL in file is incorret */
  if(!strcmp(szBool, "true") || !strcmp(szBool, "TRUE"))
  {
    iReturnValue = 1;
  }
  else if(!strcmp(szBool, "false") || !strcmp(szBool, "FALSE"))
  {
    iReturnValue = 0;
  }
  else
  {
    fprintf(stderr, "E: Invalid value of colored log level in file %s!\n",
gszConfFileName);

    iReturnValue = -4;
```

```
  }

  fclose(fpLogConfFile);
  fpLogConfFile = NULL;

  return iReturnValue;
}
```

APENDIX B – USAGE EXAMPLES

Example 1 - colored.c

```
/**
 * colored.c: Test to log library, using colored log level
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-06-16
 */

#include <stdio.h>
#include "log.h"

#define UNUSED(X) (void) X

int main(int argc, char **argv)
{
  int iColoredLogLevel;

  UNUSED(argc);
  UNUSED(argv);
  UNUSED(gbColoredLogLevel);
  UNUSED(gszLogFileName);
  UNUSED(gszConfFileName);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

  vSetConfFileName("log.conf");

  vSetLogLevel(iGetLogLevel());

  if(giDebugLevel < 0)
  {
    fprintf(stderr, "Error, giDebugLevel return value: %d!\n", giDebugLevel);

    exit(EXIT_FAILURE);
  }

  iColoredLogLevel = iGetColoredLogLevel();

  if(iColoredLogLevel == 0 || iColoredLogLevel == 1)
  {
    vSetColoredLogLevel(iColoredLogLevel);
  }

  vSetLogFileName("colored.log");

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("start %s function", __func__);
    vTraceInfo("INFO Message"      );
  }

  if(WARNING_LOG_DETAILS) vTraceWarning("WARNING Message");
  if(ERROR_LOG_DETAILS  ) vTraceError("ERROR Message"    );
  if(FATAL_LOG_DETAILS  ) vTraceFatal("FATAL Message"    );

  if(DEBUG_LOG_DETAILS  ) vTraceDebug("argc = %d", argc);
```

```
  if(TRACE_LOG_DETAILS  ) vTraceAll("%s(argc = %p, argv = %p)", __func__, &argc,
&argv);

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("End %s function", __func__);
  }

  return 0;
}
```

Example 2 debug.c

```
/**
 * debug.c: Exemple of the DEBUG_LOG_DETAILS
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-06-17
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include "log.h"

#define UNUSED(X) (void) X

/**
 * Structure that
 * represents a person
 */
typedef struct Person
{
  char szName[64];
  uint8_t uAge;
} Person, *PPerson;

static void vInitPerson(PPerson *ppstPerson)
{
  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("start %s function", __func__);
    vTraceInfo("Setting the default values in members of struct Person");
  }

  memset((*ppstPerson)->szName, 0, sizeof((*ppstPerson)->uAge));
  (*ppstPerson)->uAge = 0;

  if(DEBUG_LOG_DETAILS)
  {
    vTraceDebug("memset((*ppstPerson)->szName, 0, sizeof((*ppstPerson)-
>uAge))");
    vTraceDebug("(*ppstPerson)->uAge = 0");
  }

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("end %s function", __func__);
  }
}
```

```
static void vCreatePerson(PPerson *ppstPerson, const char *szName, const uint8_t
uAge)
{
  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("start %s function", __func__);
    vTraceInfo("create a new person");
  }

  strcpy((*ppstPerson)->szName, szName);
  (*ppstPerson)->uAge = uAge;

  if(DEBUG_LOG_DETAILS)
  {
    vTraceDebug("Values of the member Person");
    vTraceDebug("(*ppstPerson)->szName = %s", (*ppstPerson)->szName);
    vTraceDebug("(*ppstPerson)->uAge = %d", (*ppstPerson)->uAge);
  }

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("Person created with succes");
    vTraceInfo("end %s function", __func__);
  }
}

static void vShowPerson(PPerson pstPerson)
{
  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("start %s function", __func__);
    vTraceInfo("Show Person");
    vTraceInfo("Name: %s\n"
             "Age.: %d\n", pstPerson->szName,
                           pstPerson->uAge);
  }

  printf("Name: %s\n"
         "Age.: %d\n", pstPerson->szName,
                       pstPerson->uAge);

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("end %s function", __func__);
  }
}

static void vDestroyPerson(PPerson *ppstPerson)
{
  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("start %s function", __func__);
  }

  memset((*ppstPerson)->szName, 0, sizeof((*ppstPerson)->uAge));
  (*ppstPerson)->uAge = 0;

  if(DEBUG_LOG_DETAILS)
  {
    vTraceDebug("memset((*ppstPerson)->szName, 0, sizeof((*ppstPerson)-
>uAge))");
    vTraceDebug("(*ppstPerson)->uAge = 0");
    vTraceDebug("Freeing Person structure pointer");
```

```
  }

  free((*ppstPerson));
  (*ppstPerson) = NULL;

  if(DEBUG_LOG_DETAILS)
  {
    vTraceDebug("free((*ppstPerson))");
    vTraceDebug("(*ppstPerson) = NULL");
    vTraceDebug("Person structure pointer free");
  }

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("end %s function", __func__);
  }
}

int main(int argc, char **argv)
{
  PPerson pstPerson = (PPerson) malloc(sizeof(Person));

  UNUSED(argc);
  UNUSED(argv);
  UNUSED(gbColoredLogLevel);
  UNUSED(gszLogFileName);
  UNUSED(gszConfFileName);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

  vSetConfFileName("log.conf");

  vSetLogLevel(iGetLogLevel());

  if(giDebugLevel < 0)
  {
    fprintf(stderr, "Error, giDebugLevel return value: %d!\n", giDebugLevel);

    exit(EXIT_FAILURE);
  }

  vSetLogFileName("debug.log");

  /* Initial log messages */
  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("Start of function %s", __func__);
  }

  if(pstPerson == NULL)
  {
    if(DEBUG_LOG_DETAILS)
    {
      vTraceDebug("E: out memory in allocation of pstPerson!");
    }
  }

  vInitPerson(&pstPerson);

  vCreatePerson(&pstPerson, "Gustavo Bacagine", 23);

  vShowPerson(pstPerson);
```

```
  vDestroyPerson(&pstPerson);

  /* End log messages */
  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("End of function %s", __func__);
  }

  return 0;
}
```

Example 3 - hello_log.c

```
/**
 * hello_log.c
 *
 * Writeen by Gustavo Bacagine <gustavo.bacaigne@protonmail.com>
 *
 * Descritpion: Print a hello world in a log file.
 *
 * Date: 23/09/2023
 */

#include <stdio.h>
#include "log.h"

#define UNUSED(X) (void) X

int main(int argc, char **argv)
{
  UNUSED(argc);
  UNUSED(argv);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

  vSetConfFileName("log.conf");

  vSetLogLevel(iGetLogLevel());

  if(giDebugLevel < 0)
  {
    fprintf(stderr, "Error, giDebugLevel return value: %d!\n", giDebugLevel);

    exit(EXIT_FAILURE);
  }

  vSetLogFileName("hello_log.log");

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("%s - begin", __func__);
    vTraceInfo("Hello World!!!");
    vTraceInfo("%s - end", __func__);
  }

  return 0;
}
```

Example 4 - trace.c

```
/**
```

```
 * trace.c
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Description: File created to test the new macros:
 *  vTrace
 *  vTraceBegin
 *  vTraceEnd
 *
 * Date: 2023-10-17
 *
 */

#include <stdio.h>
#include "log.h"

#define UNUSED(X) (void) X

void vMyFunc(void)
{
  vTraceBegin();

  vTrace(INFO_LEVEL, "INFO MESSAGE");
  vTrace(WARNING_LEVEL, "WARNING MESSAGE");
  vTrace(ERROR_LEVEL, "ERROR MESSAGE");
  vTrace(FATAL_LEVEL, "FATAL MESSAGE");
  vTrace(DEBUG_LEVEL, "DEBUG MESSAGE");
  vTrace(TRACE_LEVEL, "TRACE MESSAGE");

  vTraceEnd();
}

int main(int argc, char **argv)
{
  UNUSED(argc);
  UNUSED(argv);
  UNUSED(gbColoredLogLevel);
  UNUSED(gszLogFileName);
  UNUSED(gszConfFileName);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

  vSetConfFileName("log.conf");

  vSetLogLevel(iGetLogLevel());

  if(giDebugLevel < 0)
  {
    fprintf(stderr, "Error, giDebugLevel return value: %d!\n", giDebugLevel);

    exit(EXIT_FAILURE);
  }

  vSetLogFileName("trace.log");

  vTraceBegin();

  vTrace(INFO_LEVEL, "Main Info Message :)");

  vMyFunc();

  vTraceEnd();
```

```
  return 0;
}
```

Example 5 - trace_level.c

```
/**
 * trace_level.c: Test to log library with TRACE_LOG_DETAILS
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-06-16
 */

#include <stdio.h>
#include "log.h"

#define UNUSED(X) (void) X

int main(int argc, char **argv, char **envp)
{
  int ii;

  UNUSED(gbColoredLogLevel);
  UNUSED(gszLogFileName);
  UNUSED(gszConfFileName);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

  vSetConfFileName("log.conf");

  vSetLogLevel(iGetLogLevel());

  if(giDebugLevel < 0)
  {
    fprintf(stderr, "Error, giDebugLevel return value: %d!\n", giDebugLevel);

    exit(EXIT_FAILURE);
  }

  vSetLogFileName("trace_level.log");

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("start %s function", __func__);
    vTraceInfo("INFO Message"      );
  }

  if(WARNING_LOG_DETAILS) vTraceWarning("WARNING Message");
  if(ERROR_LOG_DETAILS  ) vTraceError("ERROR Message"    );
  if(FATAL_LOG_DETAILS  ) vTraceFatal("FATAL Message"    );

  if(DEBUG_LOG_DETAILS  )
  {
    vTraceDebug("argc = %d", argc);
    for(ii = 0; ii < argc; ii++)
    {
      vTraceDebug("argv[%d] = %s", ii, argv[ii]);
    }

    for(ii = 0; envp[ii] != NULL; ii++)
    {
      vTraceDebug("envp[%d] = %s", ii, envp[ii]);
```

```
    }
  }

  if(TRACE_LOG_DETAILS  )
  {
    vTraceAll("TRACE Message");
    vTraceAll("%s(argc=%p, argv=%p, envp=%p)", __func__, &argc, &argv, &envp);
  }

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("End %s function", __func__);
  }

  return 0;
}
```

Example 6 - Modular programming

module1.c:

```
/**
 * module1.c: Test to log library, using modular programming
 *
 * This file is module 1 of 2
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-09-14
 */

#include <stdio.h>
#include "log.h"

#define UNUSED(X) (void) X

void vShowDebugLevel(void);
void vShowColoredLogLevel(void);
void vShowConfFileName(void);
void vShowLogFileName(void);

int main(int argc, char **argv)
{
  int iColoredLogLevel;

  UNUSED(argc);
  UNUSED(argv);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

  vSetConfFileName("log.conf");

  vSetLogLevel(iGetLogLevel());

  if(giDebugLevel < 0)
  {
    fprintf(stderr, "Error, giDebugLevel return value: %d!\n", giDebugLevel);
```

```
    exit(EXIT_FAILURE);
  }

  iColoredLogLevel = iGetColoredLogLevel();

  if(iColoredLogLevel == 0 || iColoredLogLevel == 1)
  {
    vSetColoredLogLevel(iColoredLogLevel);
  }

  vSetLogFileName("module.log");

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("Start %s function", __func__);
    vTraceInfo("Debug Level = %d", giDebugLevel);
    vTraceInfo("Colored Log Level = %s",
      gbColoredLogLevel == true ? "true" : "false");
    vTraceInfo("Configure file: %s", gszConfFileName);
    vTraceInfo("Log file: %s", gszLogFileName);
  }

  vShowDebugLevel();
  vShowColoredLogLevel();
  vShowConfFileName();
  vShowLogFileName();

  if(INFO_LOG_DETAILS) vTraceInfo("End %s funciton", __func__);

  return 0;
}
```

module2.c:

```
/**
 * module2.c: Test to log library, using modular programming
 *
 * This file is module 2 of 2
 *
 * Written by Gustavo Bacagine <gustavo.bacagine@protonmail.com>
 *
 * Date: 2023-09-14
 */

#include "log.h"

#define UNUSED(X) (void) X

void vShowDebugLevel(void)
{
  UNUSED(gbColoredLogLevel);
  UNUSED(gszLogFileName);
  UNUSED(gszConfFileName);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("Start %s function", __func__);
    vTraceInfo("Debug Level = %d", giDebugLevel);
  }
```

```
  printf("Debug Level = %d\n", giDebugLevel);

  if(INFO_LOG_DETAILS) vTraceInfo("End %s funciton", __func__);
}

void vShowColoredLogLevel(void)
{
  UNUSED(gszLogFileName);
  UNUSED(gszConfFileName);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("Start %s function", __func__);
    vTraceInfo("Colored Log Level = %s",
      gbColoredLogLevel == true ? "true" : "false");
  }

  printf("Colored Log Level = %s\n",
      gbColoredLogLevel == true ? "true" : "false");

  if(INFO_LOG_DETAILS) vTraceInfo("End %s funciton", __func__);
}

void vShowConfFileName(void)
{
  UNUSED(gbColoredLogLevel);
  UNUSED(gszLogFileName);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

  if(INFO_LOG_DETAILS)
  {
    vTraceInfo("Start %s function", __func__);
    vTraceInfo("Configure file: %s", gszConfFileName);
  }

  printf("Configure file: %s\n", gszConfFileName);

  if(INFO_LOG_DETAILS) vTraceInfo("End %s funciton", __func__);
}

void vShowLogFileName(void)
{
  UNUSED(gbColoredLogLevel);
  UNUSED(gszConfFileName);
  UNUSED(kszLogLevelColorEnd);
  UNUSED(kszLogLevelColorInit);
  UNUSED(kszLogLevel);

 if(INFO_LOG_DETAILS)
  {
    vTraceInfo("Start %s function", __func__);
    vTraceInfo("Log file: %s", gszLogFileName);
  }

  printf("Log file: %s\n", gszLogFileName);

  if(INFO_LOG_DETAILS) vTraceInfo("End %s funciton", __func__);
}
```

APENDIX C – LICENSE

GNU GENERAL PUBLIC LICENSE Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>
Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

PREAMBLE

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program--to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to
your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps:
(1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.
For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If

such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the
public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your
rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code;
keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

a) The work must carry prominent notices stating that you modified it, and giving a relevant date.

b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".

c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.

b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.

c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.

d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.

e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the
Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately
under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

e) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or

b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or

c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or

d) Limiting the use for publicity purposes of names of licensors or authors of the material; or

e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or

f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains
a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means
prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do
not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a

covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever
licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that
any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a
consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the
patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that
country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of

the covered work, then the patent license you grant is automatically extended to all recipients of the covered
work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is
in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work
conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version

number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

> <one line to give the program's name and a brief idea of what it does.>
> Copyright (C) <year> <name of author>
>
> This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.
>
> This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.
>
> You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

> <program> Copyright (C) <year> <name of author>
> This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
> This is free software, and you are welcome to redistribute it under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary.
For more information on this, and how to apply and follow the GNU GPL, see <https://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <https://www.gnu.org/licenses/why-not-lgpl.html>.